Lecture 02 Sequential Circuit :

Combinational circuit:

Output is only a function of the current input values.

輸出只跟現在的輸入有關。

Sequential circuit:

Output depends not only on the current input values, but also on preceding input values.

輸出不只跟現在的輸入有關，還跟之前的輸入有關。

Avoid latches in combinational circuit. (避免不完整的 if else、case)

Avoid combinational feedbacks. (避免 a=a+1 這種東西)

Flip-flop: data storage element with 4 states (0,1, X, Z)

0: logic low

1: logic high

X: unknown, may be a 0,1, Z, or in transition

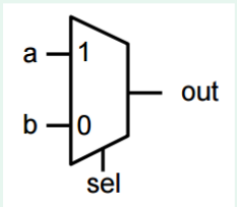
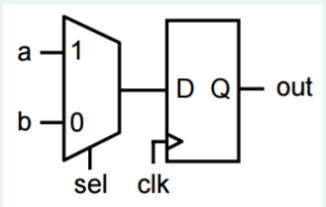
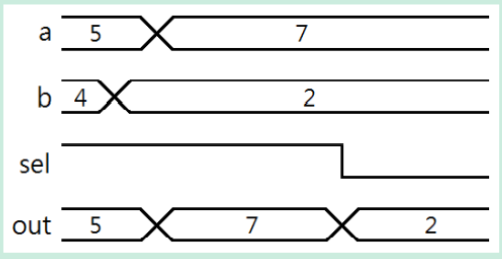
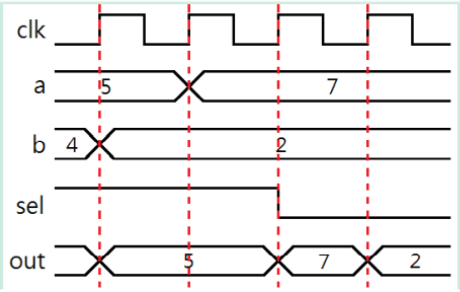
Z: high impedance, floating state

Most computations are done by combinational circuit.

Sequential elements are used for storage.

Combinational	Sequential

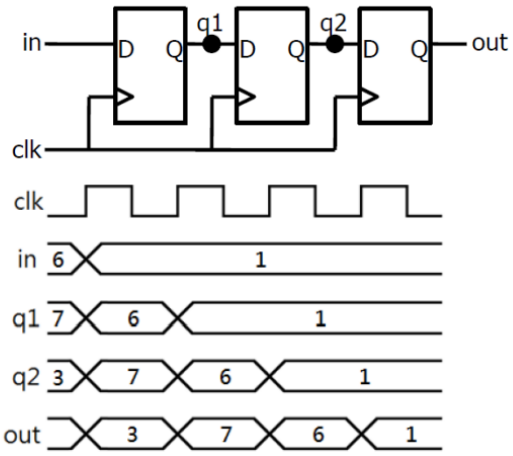
`always@(*)`
`begin`
`  if(sel) out = a;`
`  else  out = b;`
`end`
	`always@(posedge clk)`
	`begin`
	`  if(sel) out <= a;`
	`  else  out <= b;`
	`end`

Non-blocking assignment – Evaluations and assignments are executed at the same time without regard to orders or dependence upon each other.
(assignment 同時執行)

✓ **Example**

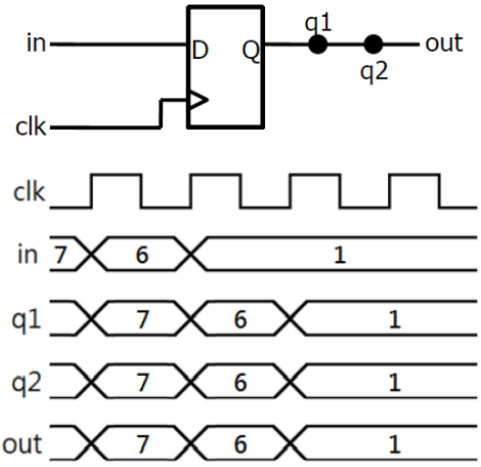
```
always @(posedge clk)
begin
   q1 <= in;
   q2 <= q1;
   out <= q2;
end
```



Blocking assignment – Evaluations and assignments are immediate and in order.
(assignment 按照順序執行)

✓ **Example**

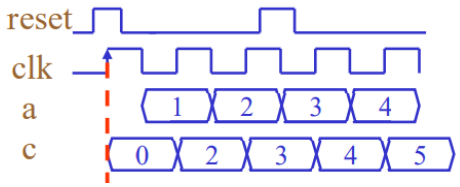
```
always @(posedge clk)
begin
   q1 = in;
   q2 = q1;
   out = q2;
end
```



Sequential blocks should only use "<=" assignments.
Combinational blocks should only use "=" assignments.

Register with **synchronous reset [2019_Spring_Mid]**

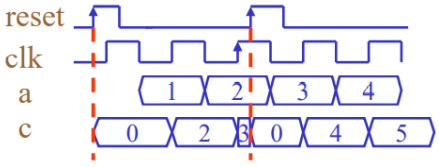
```
always @(posedge clk) begin
    if (reset) c <= 0;
    else c <= a+1;
end
```



Advantages – **Glitch filtering from reset combinational logic –Easier for ECO**
Disadvantages – **Can't be reset without clock signal** – May need a pulse stretcher • Guarantee a reset pulse wide enough – Larger area – **Increasing critical path**.

Register with **asynchronous reset [2019_Spring_Mid]**

```
always @(posedge clk or posedge reset)
begin
    if (reset) c <= 0;
    else c <= a+1;
end
```



Advantages – **Reset is independent of clock signal** – **Reset is immediate** – **Less area**
Disadvantages – **Noisy reset line could cause unwanted reset** – Metastability – **Difficult for ECO**

Q:為什麼要 reset?
A:避免 Unknown 訊號。

Coding Styles:

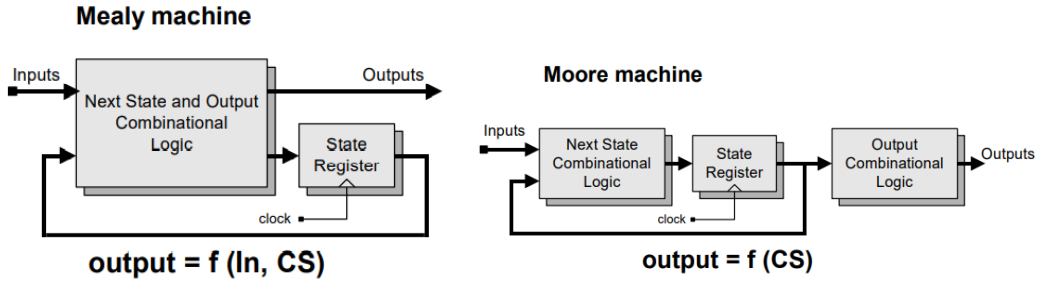
1. Naming should be readable.
2. Synthesizable codes – assign, always block, called sub-modules, if-then-else, cases, parameters, operators.
3. Data has to be described in one always block.
4. Always block can't exist both blocking and non-blocking assignment.
5. Do not put many variables in one always block
6. Use FSM (Finite State Machine)

Finite state machine

1. Powerful model for describing a sequential circuit
2. Divide a sequential circuit operation into finite number of states.
3. A state machine controller can output results depending on the input signal, control signal and states.
4. As different input or control signal changes, the state machine will take a proper

state transition.

Mealy machine – The outputs depend on the current state and inputs.



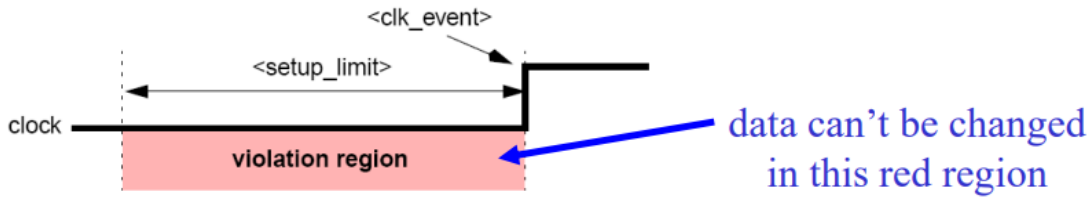
Moore machine – The outputs depend on the current state only.

FSM separate current state, next state and output logic.

Setup time check – The $setup system task determines whether a data signal remains stable for a minimum specified time before a transition in an enabling, such as a clock event.

在 clock edge 前，input signal 所需要穩定的時間。

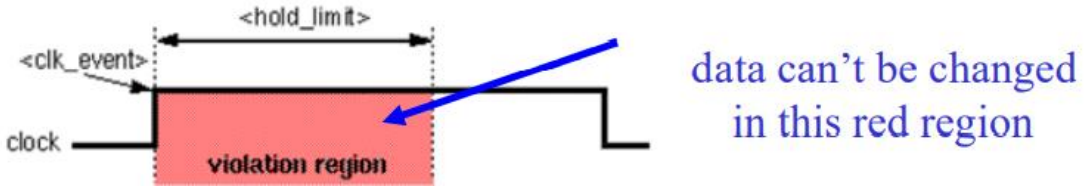
The time that the input signal must be stabilized before the clock edge.



Hold time check – The $hold system task determines whether a data signal remains stable for a minimum specified time after a transition in an enabling signal, such as a clock event.

在 clock edge 後，input signal 所需要穩定的時間。

The time that the input signal must be stabilized after the clock edge.



Timing report: setup time

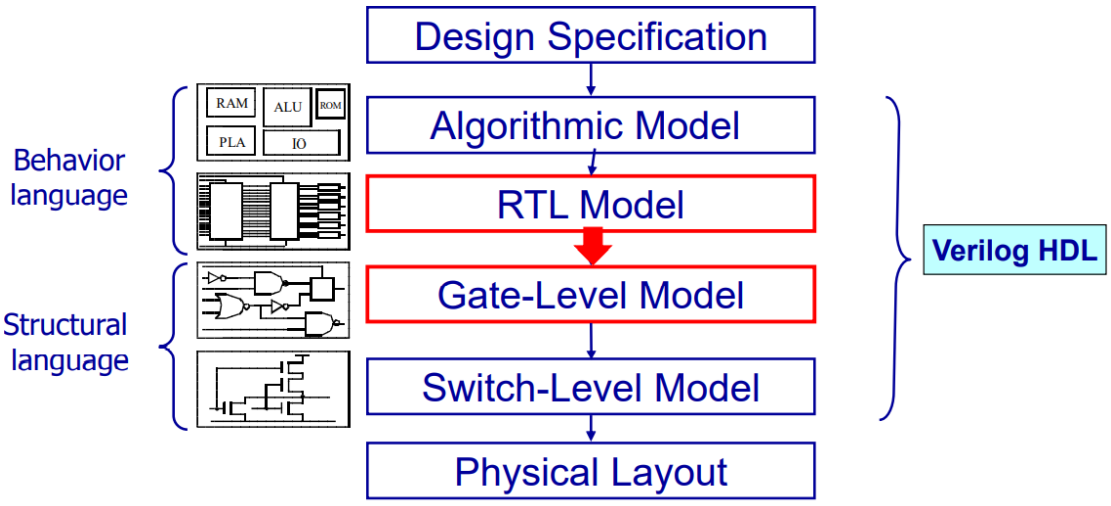
```
clock CLK_1 (rise edge)                  2.00       2.00
clock network delay (ideal)              2.00       4.00
clock uncertainty                       -0.50       3.50
IN_A_reg[0]/CK (EDFFXL)                  0.00       3.50 r
library setup time                      -0.42       3.08
data required time                                  3.08
-----------------------------------------------------------
data required time                                  3.08
data arrival time                                  -3.08
-----------------------------------------------------------
slack (MET)                                         0.00
```

Timing report: hold time

Slacks should be MET!
(non-negative)

```
clock CLK_2 (rise edge)                  0.00       0.00
clock network delay (ideal)              4.00       4.00
clock uncertainty                        1.00       5.00
IN_B_reg[20]/CK (EDFFXL)                 0.00       5.00 r
library hold time                       -0.19       4.81
data required time                                  4.81
-----------------------------------------------------------
data required time                                  4.81
data arrival time                                  -4.82
-----------------------------------------------------------
slack (MET)                                         0.01
```

Design Flow



Logic synthesis

1. A process by which behavioral model of a circuit is turned into an implementation in terms of logic gates. (把 RTL 轉成 Gate level)
2. Synthesis = Translation + Mapping + Optimization

Design compiler

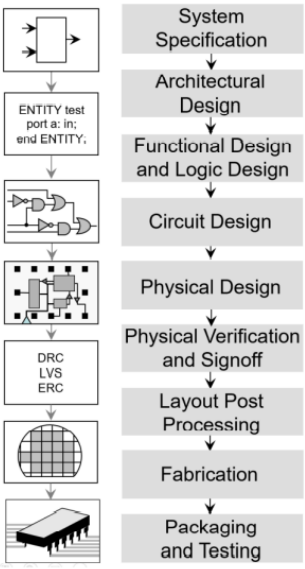
1. A tool by Synopsys, Inc. that synthesizes your HDL designs (Verilog) into optimized technology-dependent, gate-level designs.
2. It can **optimize both combinational and sequential designs for speed, area, and power.**

Lecture 03 Testbench and Pattern :

Verification == Bug Hunting – A process in which a design is verified against a given design specification before tape out.

Verification include:

1. Functionality (Main goal !!)
2. Performance
3. Power
4. Security
5. Safety



Steps of verification

1. Generate stimulus
2. Apply stimulus to DUT (Design Under Test)
3. Capture the response
4. Check for correctness
5. Measure progress against overall verification goal

TESTBED.v – Connecting testbench and design modules – Dump waveform

DESIGN.v – Design under test (DUT)

PATTERN.v – Pattern – Test program

Pattern

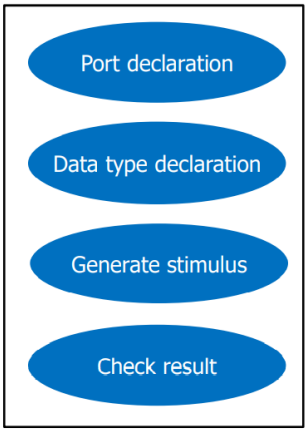
Two kinds of strategies

1. Directed Testing -> Check what you know
2. Random Testing -> Find what you don’t know

Elements of pattern file

1. Generate stimulus
2. File I/O
3. Procedural Blocks
4. Display information
5. Control flow
    - for, while, repeat, if, case, forever
6. Task and Function

PATTERN.v



Generate Stimulus

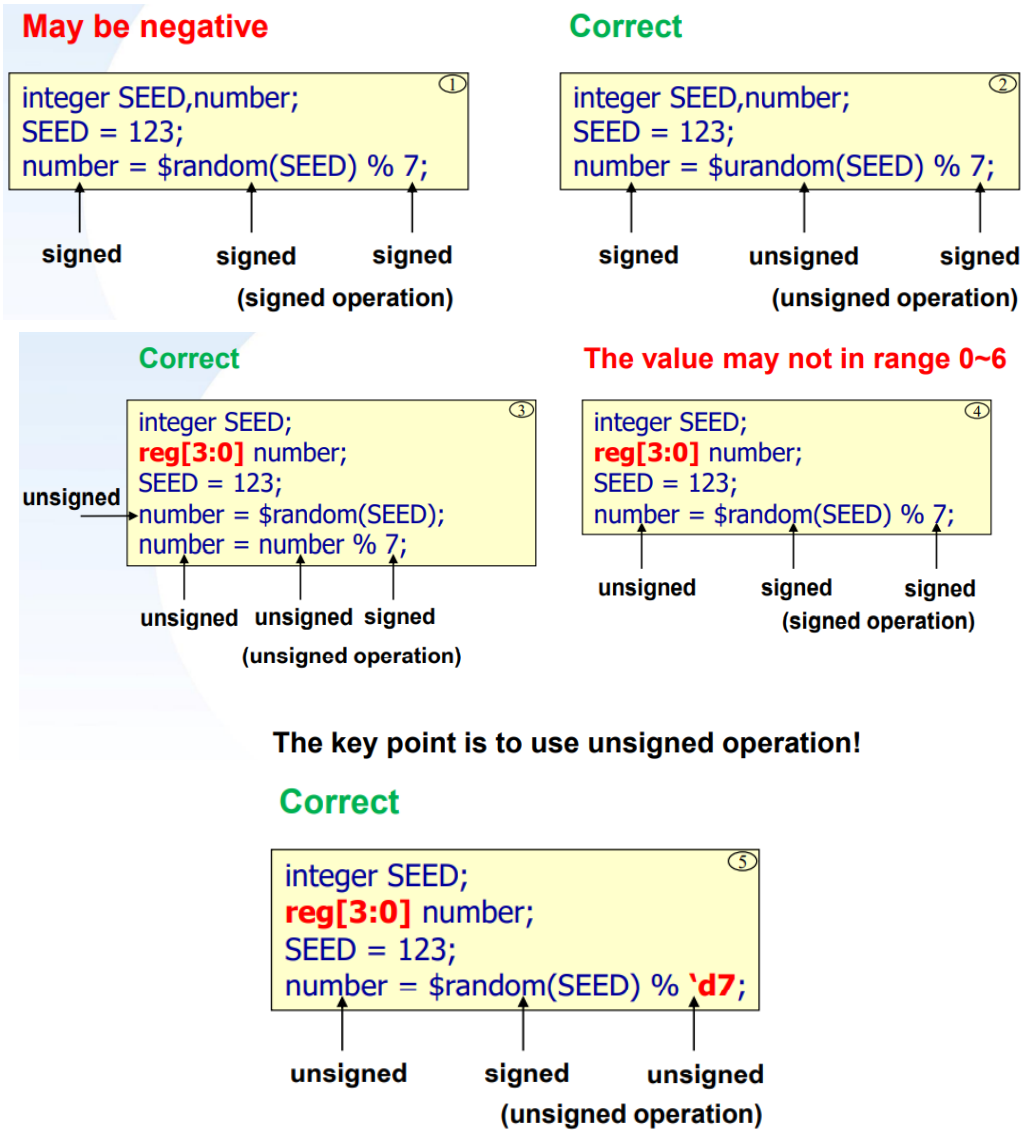
Using Verilog random system task

1. $random(seed);
    - Return 32-bit signed value
    - Seed is optional
2. $urandom(seed);
    - Return 32-bit unsigned value
    - Seed is optional
3. $urandom_range(int unsigned MAX, int unsigned MIN=0);
    - Return value inside range

Using high level language with file IO

A simple example

等號右邊有 unsigned，就是 unsigned operation。



File I/O

Open file

$fopen opens the specified file and returns a 32-bit descriptor.

file_descriptor = $fopen(“file_name” , “type”);

file_descriptor: bit 32 always be set (=1), remaining bits hold a small number indicating what file is opened.

type: “r” , open for read; “w”, open for write

multi_channel_descriptor = $fopen(“file_name”);

Multi_channel_descriptor: bit 32 always be clear (=0), bit 0 represent standard output, each remaining bit represents a single output channel.

Close file

$fclose system task closes the channels specified in the multichannel descriptor

$fclose();

The $fopen task will reuse channels that have been closed

File Input

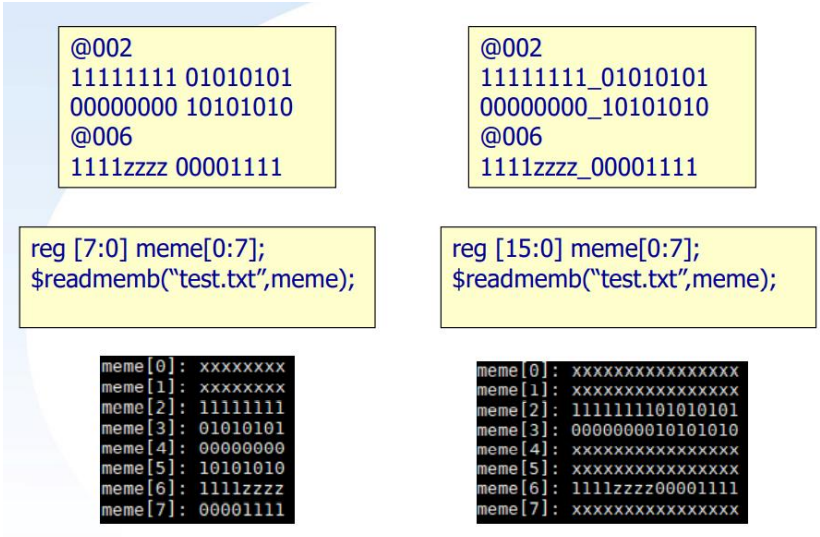
$fscanf – reading formatted data

i = $fscanf(," text", signal ,signal,…);

$readmemb, $readmemh

$readmemb("file_name", memory_name [, start_address [, end_address]]);

$readmemh("file_name", memory_name [, start_address [, end_address]]);



File Output

$fdisplay, $fwrite, $fstrobe, $fmonitor

$fdisplay (,["format_specifiers",]);

$fwrite (,["format_specifiers",]);

$fstrobe (,["format_specifiers",]);

$fmonitor (,["format_specifiers",]);

File I/O Example

```
module PATTERN(
    output reg rst_n,
    output reg [7:0] opa,
    output reg [7:0] opb,
    input clk,
    input  [7:0] sum,
    input ca,
    );

integer f_in, f_out, rc;
initial begin
  f_in  = $fopen("../00_TESTBED/in.txt", "r");
  f_out = $fopen("../00_TESTBED/out.txt", "w");
end

initial begin
    rst_n = 1;
    opa = 0;
    opb = 0;
    repeat(5) @(posedge clk);
    rst_n = 0;
    repeat(5) @(posedge clk);
    rst_n = 1;
    rc = $fscanf(f_in,"opa=%d,opb=%d",opa,opb);
    @(posedge clk);
    $fdisplay(f_out,"%d + %d = %d",opa,opb,sum);
    repeat(5) @(posedge clk);
    $display("Adder Simulation End!");
    $finish;
end
endmodule
```

Procedural Blocks

All procedural blocks will be executed concurrently.

Initial Blocks

Only be executed once

Always Blocks

Will be executed if the condition is met

- If simulation never stop, check…

1. No "$finish" in pattern
2. Have combinational loop in design
3. Have loop in pattern

Task and Function

Task

A task is typically used to perform debugging operations, or to behaviorally describe hardware.

1. Tasks may execute in non-zero simulation time
2. Can have timing controls (#delay, @, wait).
3. Can have port arguments (input, output, and inout) or none.
4. Can enable task or function.
5. Does not return a value.
6. Not synthesizable.
7. Task can take, drive and source global variables, when no local variables are used.

Function

A function is typically used to perform a computation, or to represent combinational logics.

1. always execute in 0 simulation time
2. Can't have timing controls.
3. Has only input arguments and returns a single value through the function name.
4. Can enable function, but can't enable task.
5. Returns a single value.
6. Synthesizable

There are mainly four kinds of instruction to **display information**.

$display: automatically prints a new line to the end of its output

$display ([“format_specifiers”,]);

Asynchronous Reset and Clock

Asynchronous reset:

Reset signal will reset all registers on the falling edge of reset signal.

Clock signal

Clock signal should be forced to 0 before reset signal is given.

Using always procedure to produce a duty cycle 50% clock signal

Check output data when out_valid is high.

Testbench

Encapsulate DESIGN.v and PATTERN.v to be a top verification file

Key element

1. Timescale
2. Dump Waveform
3. Port Connection

‘timescale

`timescale <time_unit>/<time_precision>

CYCLE/2.0 = CYCLE/2 ??
If CYCLE = 5
◆ CYCLE/2.0 = 2.5
◆ CYCLE/2 = 3

CYCLE/2.0 = 1.25 ns
Precision requirement: 0.01ns < 100ps
Precision loss !! The CYCLE/2.0 will become 1.3ns

‘timescale Unit/Precision	Delay	Time Delay
‘timescale 10ns/1ns	#5	50ns
‘timescale 10ns/1ns	#5.738	57ns
‘timescale 10ns/10ns	#5.5	60ns
‘timescale 10ns/100ps	#5.738	57.4ns

Dump Waveform

Used in **RTL** simulation or **gate-level** simulation

Dump wave form in fsdb format for viewing in **nWave**

Include timing information in the simulation

```
initial begin
        `ifdef RTL
                $fsdbDumpfile("Design.fsdb");
                $fsdbDumpvars(0,"+mda");
        `endif
        `ifdef GATE
                $fsdbDumpfile("Design_SYN.fsdb");
                $fsdbDumpvars(0,"+mda");
                $sdf_annotate("CORE_SYN.sdf",dut);
        `endif
end
```

Port Connection

The input and output is reverse between design.v and pattern.v.

Lecture 04 Advanced Sequential Circuit Design:

Timing Issue

Setup time ($tsetup$)

The time that the input signal must be stabilized before the clock edge.

Hold time ($thold$)

The time that the input signal must be stabilized after the clock edge.

Contamination delay

The minimum amount of time from an input changes until any output starts to change its value.

Clk-to-Q contamination delay ($tccq$)

The minimum amount of time from an clock edge until Q starts to change its value.

Logic contamination delay (tcd)

The minimum amount of time from a combinational logic input (Q_1) changes until combinational logic output (D_2) starts to change its value.

Propagation delay

The maximum amount of time from input changes until all output reaches steady state.

Clk-to-Q propagation delay ($tpcq$)

The maximum amount of time from an clock edge until Q reaches steady state.

Logic propagation delay (tpd)

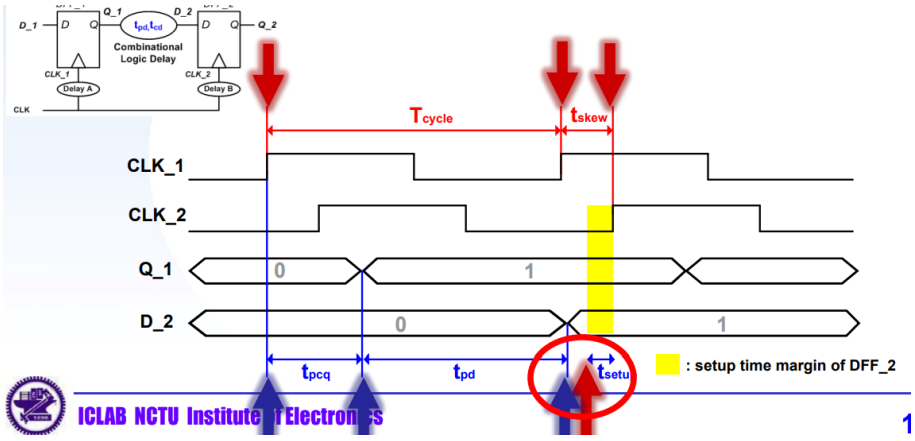
The maximum amount of time from a combinational logic input (Q_1) changes until combinational logic output (D_2) reaches steady state.

Setup time criterion: $(Tcycle + tskew) > (tpcq + tpd + tsetup)$

data required time = $Tcycle + tskew - tsetup$

data arrival time = $tpcq + tpd$

Slack = data required time - data arrival time

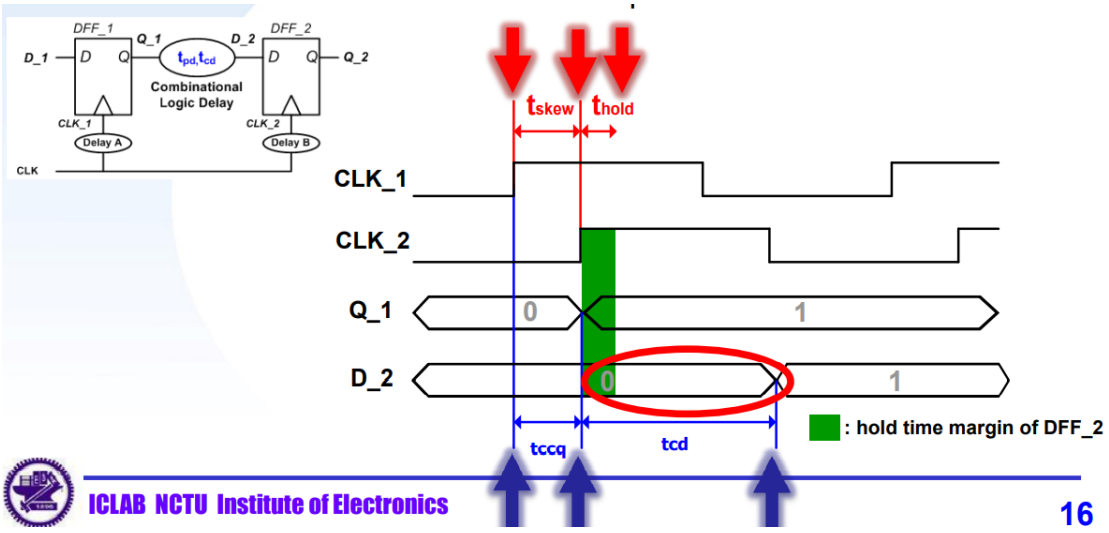


Hold time criterion: $(tccq + tcd) > (thold + tskew)$

data required time = $tskew + thold$

data arrival time = $tccq + tcd$

Slack = data arrival time – data required time



Setup violation =>too many works in one cycle

Apply pipelining。Increase clock period for setup violation。

Hold violation =>insufficient delay

add delays to the violated path, such as buffers/inverters/Muxes

IP (Intellectual Property)

Hard IP : GDSII format, high performance but technology dependent.

Firm IP : Netlist resource, less used.

Soft IP : RTL design, requires verification.

Usage of DesignWare Building Block IP [2019_Spring_Mid]

Operator inference

Advantage:

Use the HDL operator in description.

Convenient

Disadvantage:

sometimes it is inefficient when synthesizing.

Supply default function only, can not use special function.

Instantiate IP

Use SYNOPSYS design compiler shell script.

Supply different architecture for implementation. (可以提供不同架構給大家實現)

Applying pre-compiling sub-blocks speeds up the synthesis for large design.

Need to include a reference to the synthetic module in HDL code.

Synthetic Operators	HDL Operator
adder	**+, +1**
subtractor	**-, -1**
comparator	**==, <, <=, >, >=**
multiplier	*****
selector	**If, case**

最後在 Lab04 上實作 include IP。

Lecture 05 Introduction to Macros and SRAM:

Intellectual Property (IP) core

1. IP is a design of a logic function that specifies how the elements are interconnected.(像是開根號電路)
2. A designer can develop more quickly by applying IPs
3. IPs may be licensed to another party

Soft macro(IP): Synthesizable RTL

Portable and Editable

Unpredictable in terms of performance, timing, area, or power

IP protection risks

Firm macro(IP): Netlist format

Performance optimization under a specific fabrication technology(靠製程提升效能)

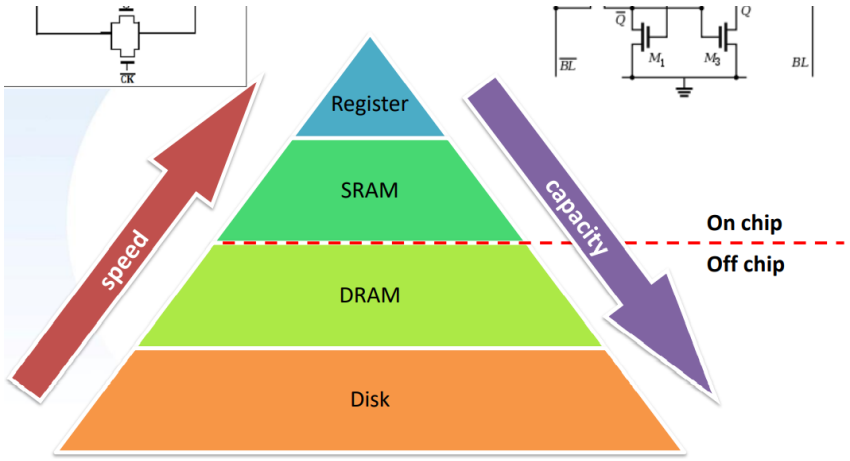
Need not synthesizing (sometimes it's time wasting)(不須合成)

Hard macro(IP): Hardware (LEF, GDS2 file format)(已經 layout 好了)

Specifies the physical pathways and wiring (proved under specific tech.)

Moving, rotating, flipping freedom but can't touch the interior (APR)

Memory Hierarchy



SRAM

Read and Write Data only

Memory has less area than register

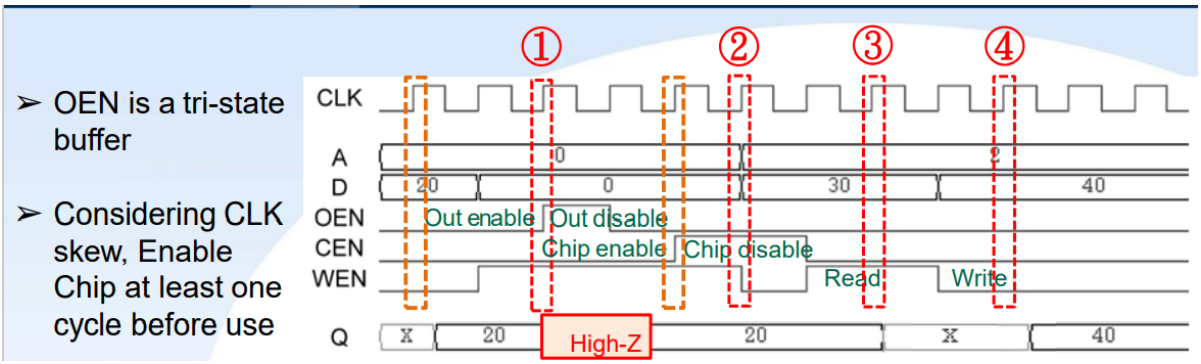
Memory is slower than register

Only one address can be accessed in the same time (single port SRAM vs. dual port)

Single port SRAM I/O Description

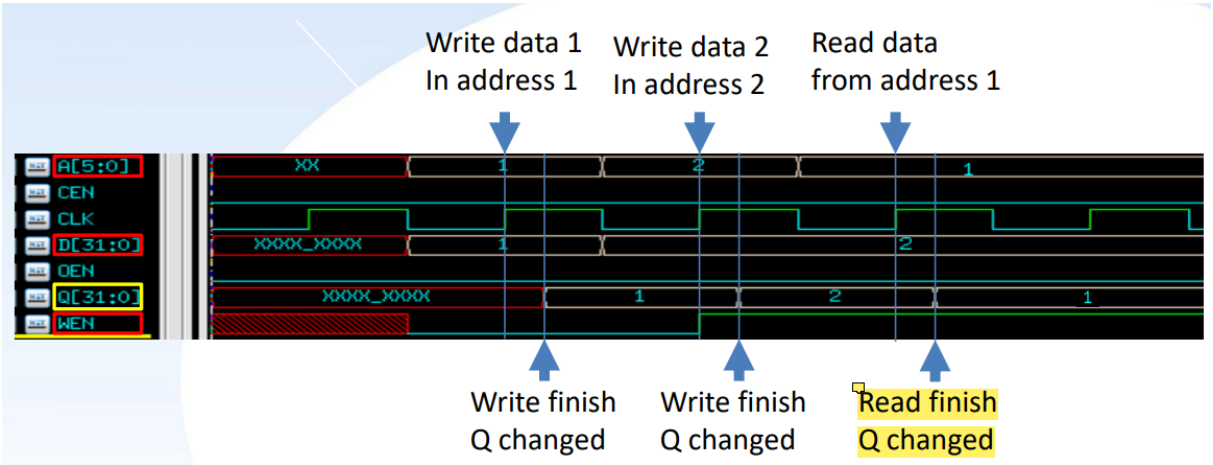
Pin	Description
A[9:0]	Address(A[0]=LSB)
D[7:0]	Data input(D[0]=LSB)
CLK	Clock input
CEN	Chip Enable Negative
WEN	Write Enable Negative
OEN	Output Enable Negative
Q[7:0]	Data Output(Q[0]=LSB)

SRAM Logic Table



SRAM Logic Table

	CEN	WEN	OEN	Data Out	Mode	Function
①	X	X	H	Z	High-Z	The data output bus Q[n-1:0] is placed in a high impedance state. Other memory operations are unaffected.
②	H	X	L	Last Data	Standby	Address inputs are disabled; data stored in the memory is retained, but the memory cannot be accessed for new reads or writes. Data outputs remain stable.
③	L	H	L	SRAM Data	Read	Data on the data output bus Q[n-1:0] is read from the memory location specified on the address bus A[m-1:0].
④	L	L	L	Data In	Write	Data on the data input bus D[n-1:0] is written to the memory location specified on the address bus A[m-1:0], and driven through to the data output bus Q[n-1:0].



Design Tips

To avoid critical path causing timing violation

Add registers after the hard macro

Use enable signal to control output register to avoid reading unknown value

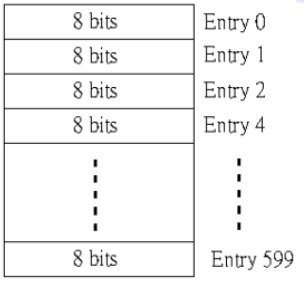
If a memory macro is used in your design, the timescale should be set according to the timescale specified by memory file. (design.v 的 timescale 要和 memory 一樣)

Memory generation example

Example :

Number of Words : 600

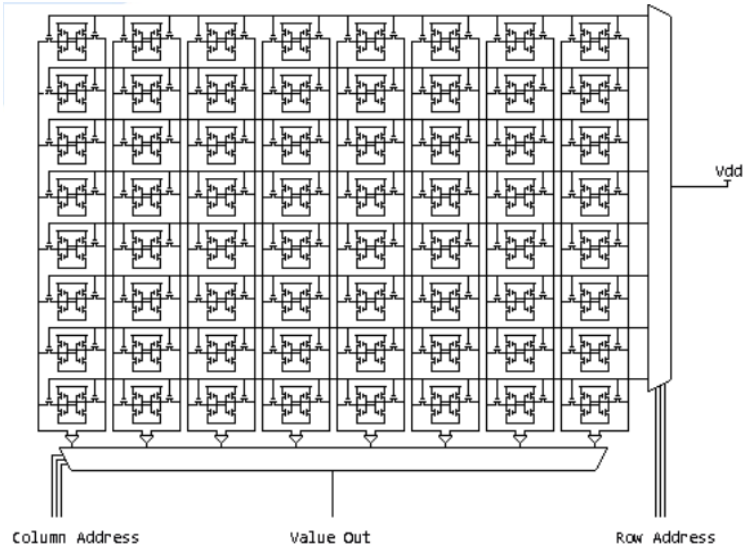
Number of Bits : 8



上述例子各個 pins 的 Bits 數

D [7:0] - Q [7:0] - A [9:0]

Memory Architectures



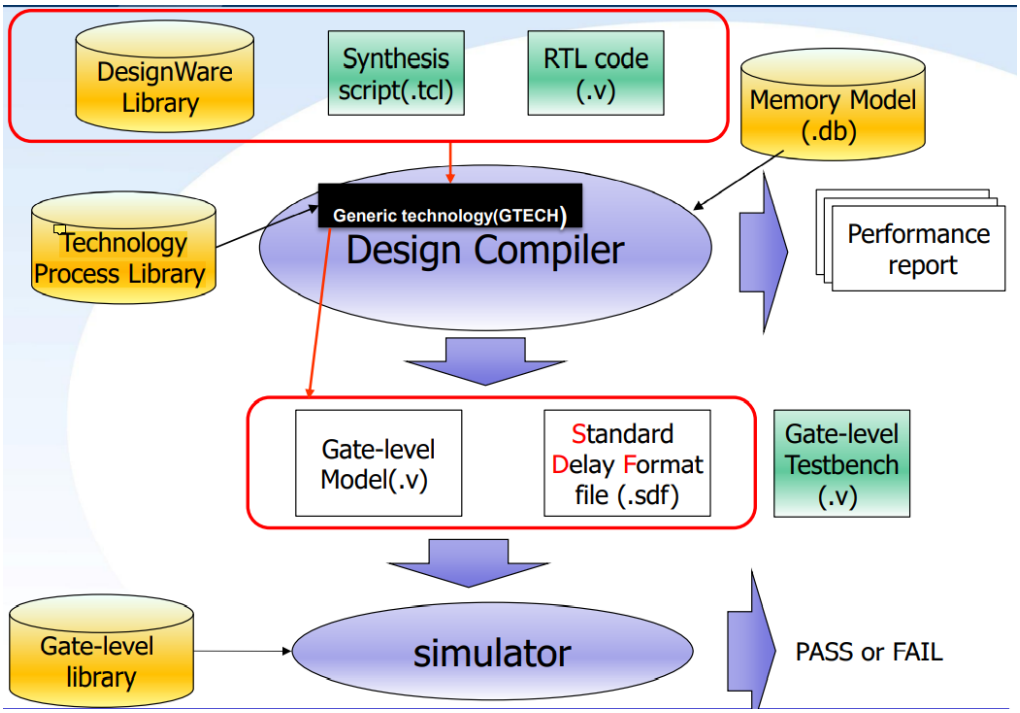
Hint: change multiplexer width to make footprint close to square.

➢ $(bit \times Mux\ Width) \approx (Words \div Mux\ Width)$ $Mux\ Width2 \approx Words \div bit$

Lecture 06 Introduction to Synthesis Flow with Synopsys Design Compiler:

Review : IC design flow

Data flow in Design Compiler



Design compiler

It synthesizes your HDL designs (Verilog) into optimized technology dependent(D35,U18…), **gate-level** designs.

It can optimize both combinational and sequential designs for speed, area, and power.

Design Compiler command-line interface

dctcl mode : Applying a script based on tool command language (tcl) which packages all the commands needed to implement specific Design Compiler functionality.

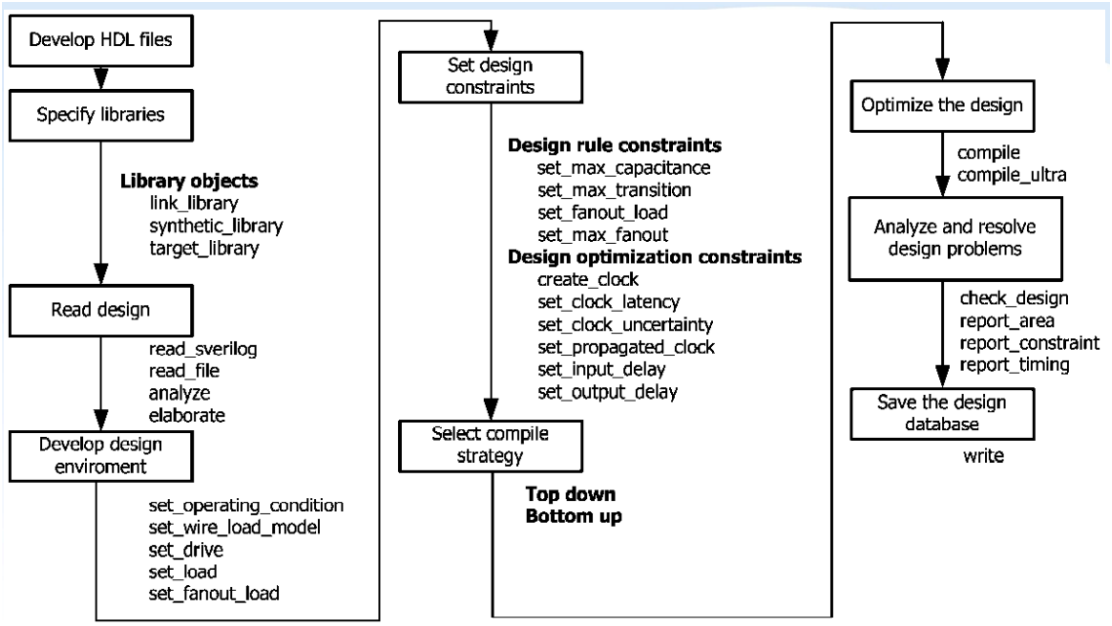
%dc_shell-t

Synthesize by Design Compiler

1. Prepare RTL code(.v) and script file(.tcl)
2. Invoke design compiler
   – Command : %dc_shell-t -f script_filename.tcl |tee log_filename.log
   – Example : %dc_shell-t -f syn.tcl |tee syn.log → ./01_run_dc
   – -f :format
   – tee: store the result into file and print on the screen
3. Check log file(.log) to see if there are error(ex: Latch) messages
4. Check report file(.report) to see if timing or area are met
5. Run gate-level simulation

Gate level netlist 裡沒有 always block 和 assign

Basic Synthesis Flow



Specify libraries

1. Synthetic Library
   • Specifies additional DesignWare libraries for optimization purposes.
   • Efficient implementations for adders, comparators, multipliers
2. Link Library
   • Specifies a list of libraries that Design Compiler can use to resolve design references.
3. Target Library
   • During synthesis, compiler selects gates from target library.
   • It also calculates the timing of the circuit, using the vendor-supplied (UMC, TSMC …) timing data of the lib.

```
set search_path {./../01_RTL \
                 ~iclabta01/umc018/Synthesis/ \
                 /usr/synthesis/libraries/syn/ \
                 /usr/synthesis/dw/ }
① set synthetic_library {dw_foundation.sldb}
② set link_library {* dw_foundation.sldb standard.sldb slow.db}
③ set target_library {slow.db}
```

Read Design

The Design Compiler optimization process works only on the designs loaded in memory.

Method 1: analyze & elaborate

Method 2: read_file ， read_verilog，read_sverilog

Method 1

```
#---------------- analyze elaborate method ------------
## this method automatically link , so we don't have to add link command
analyze -f verilog $DESIGN\.v
analyze -f verilog GENVAR_PRAC.v
elaborate $DESIGN
current_design $DESIGN
```

Method 2

```
#--------------- read_file method -----------
set hdlin_auto_save_templates TRUE
# create the intermediate file for verilog for all the later read_verilog,
# this is imporatant for reading multiple design

read_verilog GENVAR_PRAC.v
read_verilog {$DESIGN\.v}
current_design $DESIGN


#Since read_file didn't automatically link, so you must add link command
link > Report/$DESIGN\.link
```

Setting the Current Design

After the read file command, set the current design on the module you want to focus on.

Define Design Environment

Define the environment in which the design is expected to operate in by specifying **operating conditions**, **wire load models**, and **system interface characteristics**.

Defining the Operating Conditions (PVT Variations)

An operating condition is defined as a combination of **Process, Voltage, Temperature(PVT).**

Three kinds of manufacturing process models:

1. slow process model
2. typical process model
3. fast process model.

Design should be validated at the **extreme corners** (slow, fast process model) of the manufacturing process at last.

Generally, for synthesis stage, we use worst-case to ensure that the timing (setup time) is met under strict constraints.

有三個 IC 製造 model，slow、typical、fast，Design 要經過 corner 測試，也就是 slow 和 fast。看能不能符合限制條件。

Defining Wire Load Models

Before layout(APR), wire load models can be used to estimate capacitance, resistance and the area overhead due to interconnection.

Design Compiler uses physical values to calculate wire delays and circuit speeds, there are three types of wire load mode.

Top

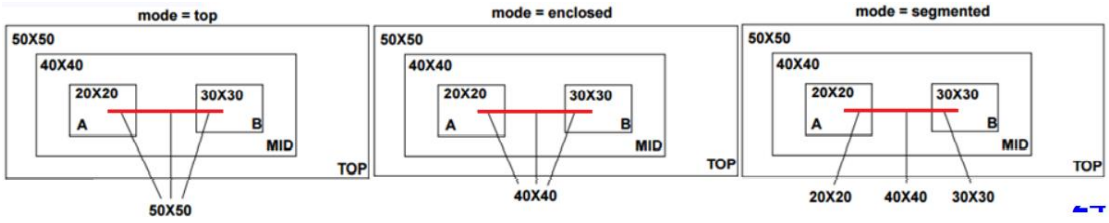
- Uses the wire load model specified for the top level of the design hierarchy for all nets

Enclosed

- Uses the wire load model of the smallest module that fully encloses the net

Segmented

- Each segment of the net gets its wire load model from the block that encompasses the net segment



Modeling the System Interface

Design Compiler supports the following ways to model the design's interaction with the external system:

• Defining drive characteristics for input ports

• Defining fan out loads on output ports

• Defining loads on input and output ports

Example : Set a load of **0.05 picofarads** on all output ports, set a drive of **1.5 kilo-ohms** on all input ports.

```
set_drive 1.5  [all_inputs]                    set_load 0.05  [all_outputs]
```

Set Design Constraints

1. Design rule constraints

   • Design rule constraints reflect technology-specific restrictions your design must meet in order to function as intended.

   • Most technology libraries specify default design rules.

   • You can apply more restrictive design rules, but you cannot apply less restrictive ones.

2. Design optimization constraints

   • User defines speed(timing) and area optimization goals for Design Compiler.

   • Speed(timing) constraints have higher priority than area.(The priority can be changed)

   • Optimization constraints are secondary to design rule constraints.

Design Optimization Constraints

Design Compiler fix hold violations at register during compilation

To fix a hold violation requires slowing down data signals.

Select Compile Strategy

Top-down compile, in which the top-level design and all its subdesigns are compiled together.

Bottom-up compile, in which the individual subdesigns are compiled separately, starting from the bottom of the hierarchy and proceeding up through the levels of the hierarchy until the top-level design is compiled.

Mixed compile, in which the top-down or bottom-up strategy, whichever is most appropriate, is applied to the individual subdesigns.

Multiple Instances of a Design Reference

In a hierarchical design, subdesigns are often referenced by more than one cell instance, that is, multiple references of the design can occur.

Use any of the following methods resolve multiple instances before running the compile command

1. The uniquify method
2. The compile-once-dont-touch method
3. The ungroup method

Uniquify (default)

The command is to duplicate and rename the multiple referenced design so that each instance references a unique design.

• "uniquify" may useless with "compile_ultra"
• Requires more memory
• Takes longer to compile

Compile-once-dont-touch method

It places the dont_touch attribute on cells, nets, references, and designs in the current design to prevent these objects from being modified or replaced during optimization.

• Compiles the reference design once
• Requires less memory and less time than the uniquify method

Ungroup

Use the ungroup command to ungroup one or more designs before optimization

• Requires more memory and takes longer to compile than the compile-once-don't-touch method
• Provides the best synthesis results
• May increase the difficulty for ECO(Engineering change order).

Compile

Default synthesis algorithm – dc_shell> **compile**

Advanced synthesis algorithm – dc_shell> **compile_ultra**

• Automatically ungroups logical hierarchies
• High-performance design
• Maximum performance
• Minimum area
• Data path

Report Analysis

```
#=====================================================
#  Output Reports
#=====================================================
report_timing >  Report/$DESIGN\.timing
report_area >  Report/$DESIGN\.area
report_resource >  Report/$DESIGN\.resource
```

Fix Glitch Suppression

When you meet glitch suppression at 03_GATE_SIM

Always occur in “dD” umc18_neg.v, line = 10043

Add “-nontcglitch” in 03_GATE ./01_run

To specify cells in the target library to be excluded during optimization

```
dc_shell> set_dont_use [get_lib_cells "slow/JKFF*"]
          set_dont_use slow/JKFF*
```

Save design

You use the write command to save the synthesized designs. Remember that Design Compiler does not automatically save designs before exiting.

Change naming rule script

Make all net and port names conform to the naming conventions for layout tool

Execute the script after compile your design

```
#=====================================================
#  Change Naming Rule
#=====================================================
set bus_inference_style "%s\[%d\]"
set bus_naming_style "%s\[%d\]"
set hdlout_internal_busses true
change_names -hierarchy -rule verilog
define_name_rules name_rule -allowed "a-z A-Z 0-9 _" -max_length 255 -type cell
define_name_rules name_rule -allowed "a-z A-Z 0-9 _[]" -max_length 255 -type net
define_name_rules name_rule -map {{"\\*cell\\*" "cell"}}
change_names -hierarchy -rules name_rule
```

Save a gate level Verilog file

Syntax : write –f verilog –o file_name.v –hierarchy

- -f : format • -o : output file name
- -hierarchy : Indicates to write all designs in the hierarchy

Post synthesis timing simulation

The post synthesis design must be simulated with estimated delays from synthesis. A SDF(standard delay format) can be generated from design compiler for this purpose. Use the following command to generate the SDF file.

Syntax : write_sdf -version sdf_version file_name

```
#=====================================================
#  Output Results
#=====================================================
set verilogout_higher_designs_first true
write -format verilog -output Netlist/$DESIGN\_SYN.v -hierarchy
write_sdf -version 3.0 -context verilog -load_delay cell Netlist/$DESIGN\_SYN.sdf -significant_digits 6
```

Modify your test file (TESTBED.v)

$sdf_annotate(“the_SDF_file_name¨, the_instance_name);

ex : $sdf_annotate(“CHIP.sdf”, I_DAG);

Generate & For Loop

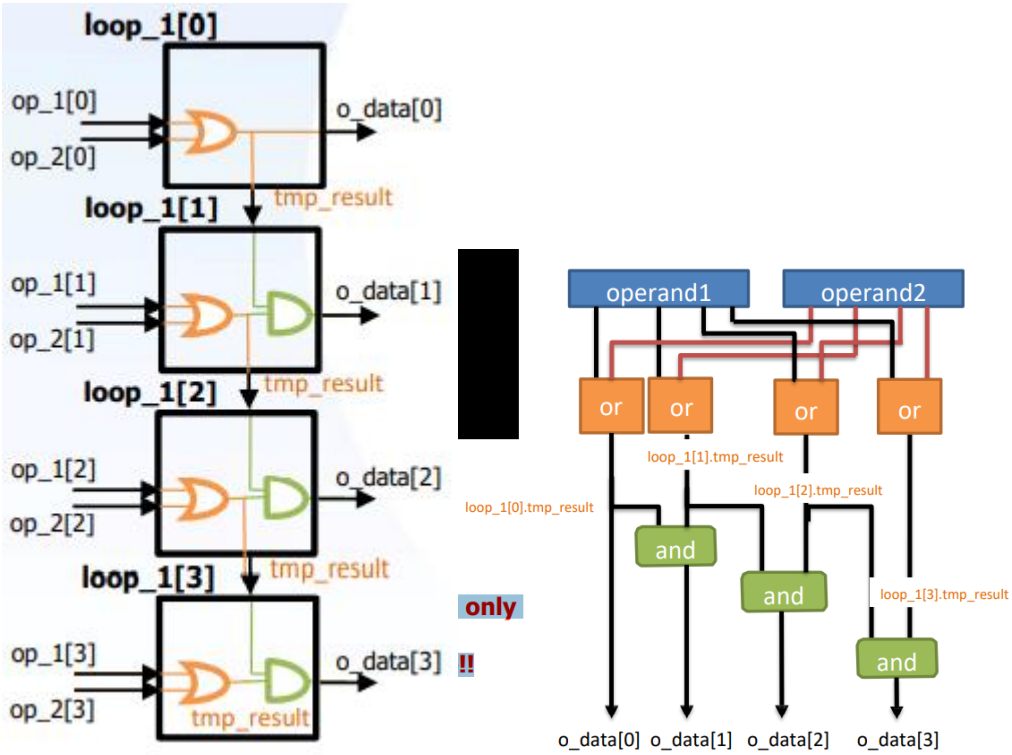
```
wire [3:0] o_data;

genvar i;
generate
for (i=0 ; i < 4; i = i+1)begin : loop_1
    wire tmp_result;
    assign tmp_result = operand1[i] | operand2[i];
    if(i == 0)begin
       assign o_data[0] = tmp_result;
    end
    else begin
       assign o_data[i] = tmp_result & loop_1[i-1].tmp_result;
    end
end
endgenerate
```

If-else in generate for can only use for select circuit but not logic determination.

How to use wire in for loop ? for_name[i].wire_name

Generate for can only use increase idx but not decrease!!



Generate blocks are useful when change the physical structure of module via parameters.

```
module top
...
adder add_8bit #(8) (.a(a0), .b(b0), .sum(sum_8bit))
adder add_16bit #(16) (.a(a1), .b(b1), .sum(sum_16bit))
...
endmodule

module adder
#(parameter LENGTH = 16)
(
input       [LENGTH-1:0]        a,
input       [LENGTH-1:0]        b,
output      [LENGTH:0]          sum
)
generate
            sum = a + b;
endgenerate
endmodule
```